

Overview

This rubric defines the detailed scoring breakdown for each deliverable. Judges will evaluate submissions against each test case independently. Partial credit is awarded where a component is attempted but incomplete or incorrect. All five deliverables are mandatory; bonus challenges do not substitute for core marks.

Deliverable	Algorithm / Method	Weight	Max Marks
1. Working CLI Tool	All algorithms + ingestion	30%	30
2. Kill Chain Report	Dijkstra output + formatting	25%	25
3. Algorithm Correctness	BFS / Dijkstra / DFS unit tests	20%	20
4. Critical Node Analysis	Graph surgery / node removal	15%	15
5. Code Quality & Docs	README, schema, readability	10%	10
TOTAL		100%	100

Deliverable 1 — Working CLI Tool [30 marks]

1.1 Data Ingestion & Graph Construction (10 marks)

The tool must correctly parse cluster-graph.json (or live kubectl output) into an in-memory directed graph. All node attributes and edge weights must be preserved and accessible during traversal.

Score Band	Criteria	Points
Full Marks	cluster-graph.json parsed without errors. All 40+ nodes and edges loaded. Node attributes (type, namespace, risk_score, is_source, is_sink, cves) and edge attributes (relationship, weight, cve, cvss) are accessible. Graph reports correct node/edge count.	10
Strong	Graph loads correctly. Minor attribute missing (e.g. cves not stored per node) or one node type not handled. All edges present.	8
Good	Graph loads but 10-20% of edges or nodes missing due to parsing logic errors. Core source/sink nodes present.	6
Partial	Graph partially constructed. Schema not fully followed. Fewer than 30 nodes loaded. Some edges missing.	4
Minimal	JSON parsed but graph library not used (e.g. stored as raw dict). No proper node/edge model.	2
Not Attempted	No graph construction. Data not loaded or tool crashes on ingestion.	0

1.2 CLI Interface & Usability (10 marks)

The tool must expose all algorithms via a clean command-line interface with named flags. Help text, error messages, and exit codes must be appropriate.

Score Band	Criteria	Points
Full Marks	All algorithms accessible via distinct CLI flags (--blast-radius, --source/--target, --cycles, --critical-node, --full-report). --help works. Unknown nodes produce clear error. Exit code 0 on success, non-zero on failure.	10
Strong	All algorithms reachable. Minor usability gap (e.g. no --help, or error message unclear for missing node).	8
Good	3 or 4 of 5 algorithm modes exposed. Positional args used instead of flags.	6
Partial	Hardcoded source/target nodes. Algorithm selected by editing code, not CLI.	3
Not Attempted	No CLI. Script runs with no arguments or only via interactive prompt.	0

1.3 End-to-End Integration Test (10 marks)

Judges will run the tool against the provided mock-cluster-graph.json and verify the combined full-report output matches expected values (see Section 6 — Test Cases).

Score Band	Criteria	Points
Full Marks	All 6 pre-planted attack paths detected. Cycle detected. Critical node identified. Report generated without exceptions. Runtime under 60 seconds on mock data.	10
Strong	5 of 6 paths detected. Cycle found. Minor formatting issue in output.	8
Good	3-4 paths detected. One algorithm produces incorrect results. No crash.	5
Partial	Tool runs but only 1-2 paths found. One or more algorithm sections missing from output.	3
Not Attempted	Tool crashes or produces no output on mock data.	0

Deliverable 2 — Kill Chain Report [25 marks]

2.1 Attack Path Accuracy (10 marks)

Each detected path must list the correct sequence of nodes, the correct hop count, and an accurate total risk score (sum of edge weights along the path). CVE annotations must appear on edges where present in the data.

Expected output for Path #1 (mock dataset):

```
dev-1 → web-frontend [CVE-2024-1234, CVSS 8.1] → sa-webapp → secret-reader → db-credentials  
→ production-db | 5 hops | Risk Score: 24.2
```

Score Band	Criteria	Points
Full Marks	All detected paths show: correct node sequence, correct hop count, correct cumulative risk score (± 0.1), CVE annotations on correct edges. Paths sorted by risk score ascending.	10
Strong	Correct paths. Risk scores off by > 0.1 due to rounding, or CVE not shown on one edge. Sort order wrong.	8
Good	Correct node sequences but hop count miscalculated (off-by-one) or risk scores computed incorrectly for 2+ paths.	5
Partial	Paths shown but node IDs used instead of names, or paths truncated. No risk score.	3
Not Attempted	No path output, or paths are fabricated/incorrect.	0

2.2 Report Readability & Structure (8 marks)

The report must be clearly structured with labelled sections, risk severity labels (CRITICAL / HIGH / MEDIUM / LOW), and actionable remediation language.

Score Band	Criteria	Points
Full Marks	Report has clearly labelled sections (Attack Paths, Blast Radius, Cycles, Critical Node, Summary). Risk severity labels applied correctly based on score thresholds. Each path includes source, target, hops, score, severity. Summary section present.	8
Good	Most sections present. Severity label missing or threshold wrong on 1-2 paths. Summary absent.	6
Partial	Report is a raw list with no section structure. No severity labels. Paths printed without hop count.	3
Not Attempted	No report output.	0

2.3 Remediation Advice (7 marks)

The report must include actionable remediation recommendations for each identified risk, not just a description of the problem.

Score Band	Criteria	Points
------------	----------	--------

Full Marks	Each attack path includes specific remediation (e.g. 'Remove RoleBinding secret-reader from sa-webapp', 'Patch CVE-2024-1234 on web-frontend', 'Break cycle: revoke admin-grant from service-b to service-a'). Critical node recommendation includes estimated impact (N paths eliminated).	7
Good	Generic remediation advice present (e.g. 'reduce privileges') but not specific to each path or node.	5
Partial	Remediation section exists but is placeholder text or duplicated for all paths.	2
Not Attempted	No remediation advice.	0

Deliverable 3 — Algorithm Correctness [20 marks]

Judges will run each algorithm against the mock dataset and three additional hidden test graphs. Correct output on the mock dataset earns base marks; correct output on hidden tests earns full marks.

3.1 Algorithm 1 — Blast Radius (BFS) [7 marks]

Test Case BFS-1: Source = pod-webfront, hops = 3

Expected: Hop 1 = {sa-webapp, sa-default, internal-api-svc, sidecar-proxy} | Hop 2 = {secret-reader, tls-cert, api-key, cluster-admin, api-server} | Hop 3 = {db-credentials, secret-admin-token, sa-worker, db-url-config}

Test Case BFS-2: Source = cicd-bot, hops = 2

Expected: Hop 1 = {sa-cicd} | Hop 2 = {deployer, cicd-deploy-token}

Test Case BFS-3 (hidden): Single-node source with no outbound edges.

Expected: Empty blast radius (0 reachable nodes).

Score Band	Criteria	Points
Full Marks	BFS-1 and BFS-2 exact match. BFS-3 (hidden) handled correctly (empty result, no crash). Layer assignments (hop number) correct. Nodes not double-counted across layers.	7
Strong	BFS-1 correct. BFS-2 off by one node. BFS-3 not tested.	5
Good	BFS runs. Hop layers merged into flat list (layering not implemented). Node set correct.	4
Partial	BFS implemented but visits nodes in wrong order or revisits nodes (no visited set). Incorrect node counts.	2
Not Attempted	BFS not implemented. DFS or random walk used instead.	0

3.2 Algorithm 2 — Dijkstra's Shortest Path [7 marks]

Test Case DIJK-1: Source = user-dev1, Target = db-production

Expected path: user-dev1 → pod-webfront → sa-webapp → role-secret-reader → secret-db-creds → db-production | Cost = 24.1

Test Case DIJK-2: Source = internet, Target = ns-kube-system

Expected path: internet → web-frontend → sa-default → cluster-admin → secret-admin-token → ns-kube-system | Cost = 32.0

Test Case DIJK-3 (hidden): Source and target with no connecting path.

Expected: Clear 'No path found' message. No exception raised.

Score Band	Criteria	Points
Full Marks	DIJK-1 and DIJK-2 return correct path and cost (± 0.05). DIJK-3 handled gracefully with informative message. Uses edge weight correctly (not hop count).	7
Strong	DIJK-1 correct. DIJK-2 finds a valid path but not the shortest. DIJK-3 not crash.	5
Good	Shortest path found by hop count (unweighted BFS) instead of Dijkstra. Results differ from expected.	3
Partial	Path returned but incorrect. Cost not calculated or always 0.	1

Not Attempted	Algorithm not implemented or always returns empty.	0
---------------	--	---

3.3 Algorithm 3 — Cycle Detection (DFS) [6 marks]

Test Case DFS-1: Full graph — mock dataset

Expected: Exactly 1 cycle detected: [svc-service-a, svc-service-b] (mutual admin-grant). No false positives.

Test Case DFS-2 (hidden): Graph with 3 planted cycles.

Expected: All 3 cycles returned. No duplicate reporting of the same cycle.

Score Band	Criteria	Points
Full Marks	DFS-1: exactly 1 cycle found, correct nodes. DFS-2 (hidden): all 3 cycles found, no duplicates. Cycle reported as ordered node list.	6
Strong	DFS-1 correct. DFS-2: 2 of 3 cycles found.	4
Good	Cycle detection works but reports duplicates (e.g. A→B→A and B→A→B counted separately).	3
Partial	Cycle detection attempted. Returns True/False (cycle exists) but not the cycle nodes.	1
Not Attempted	No cycle detection.	0

Deliverable 4 — Critical Node Analysis [15 marks]

4.1 Correct Critical Node Identification (8 marks)

The algorithm must correctly identify the single non-source, non-sink node whose removal from the graph eliminates the greatest number of source-to-sink paths. Judges will verify by independently running `nx.all_simple_paths` on the modified graph.

Expected result (mock dataset):

Critical node: web-frontend (Pod) | Paths eliminated: 32 of 46 | Runner-up: api-server (Pod), -24 paths

Score Band	Criteria	Points
Full Marks	Correct critical node identified (web-frontend). Correct elimination count (32). Top-5 ranking output matches expected ranking. Source and sink nodes correctly excluded from candidates.	8
Strong	Correct critical node. Elimination count off by ± 2 (path counting method differs slightly). Ranking mostly correct.	6
Good	Second-ranked node (api-server) returned as critical node. Methodology correct but tie-breaking logic differs.	4
Partial	Algorithm runs and returns a node, but node is clearly wrong (e.g. a source or sink node returned). No ranking.	2
Not Attempted	Analysis not implemented or crashes.	0

4.2 Methodology Correctness (7 marks)

The removal-and-recount methodology must be correctly applied: the original graph must be unmodified after each test removal, and path counting must use all simple paths (not just shortest paths).

Score Band	Criteria	Points
Full Marks	Graph copied (not mutated) before each node removal. All simple paths counted (not just shortest). Cutoff depth applied consistently. Baseline path count correct. Graph restored to original state after analysis.	7
Strong	Correct methodology. Minor issue: cutoff not applied (performance risk on large graphs) or baseline not reported.	5
Good	Graph mutated in-place (original graph corrupted after analysis). Results still numerically correct on mock data but would fail on sequential calls.	3
Partial	Only shortest path counted (not all paths). Node betweenness centrality used as a proxy (valid approach but not specification-compliant).	2
Not Attempted	No methodology. Random node returned.	0

Deliverable 5 — Code Quality & Documentation [10 marks]

5.1 Code Readability (4 marks)

Score Band	Criteria	Points
Full Marks	Functions have docstrings. Variables clearly named. Algorithms separated into distinct functions. No magic numbers. PEP-8 compliant (or equivalent for Go/Java). No dead code.	4
Good	Mostly readable. A few long functions. Some magic numbers. Minor style inconsistencies.	3
Partial	Code works but is in one long main() block. No functions. Hardcoded values throughout.	1
Not Attempted	Unreadable / obfuscated.	0

5.2 Schema Documentation (3 marks)

The cluster-graph.json schema must be documented so that a new team member could extend it without reading the source code.

Score Band	Criteria	Points
Full Marks	README or separate schema doc explains all node types, all edge relationship types, weight semantics, is_source/is_sink flags, and CVE field format. Example node and edge JSON included.	3
Good	Schema documented but missing 1-2 field explanations (e.g. weight semantics not defined, or node types not enumerated).	2
Partial	Schema mentioned but not documented. Reader must infer from JSON.	1
Not Attempted	No schema documentation.	0

5.3 README & Setup Instructions (3 marks)

Score Band	Criteria	Points
Full Marks	README includes: installation steps (pip install), all CLI usage examples with expected output snippets, description of all 4 algorithms, project structure overview. New user can run the tool within 5 minutes.	3
Good	README present with install and basic usage. Missing algorithm descriptions or project structure.	2
Partial	README is a stub with title only, or only lists file names.	1
Not Attempted	No README.	0

Bonus Challenges [up to +15 marks, non-substitutable]

Bonus marks are awarded in addition to core marks. They cannot substitute for missing core deliverables. Maximum score including bonuses is capped at 100.

Bonus	Description	Marks	Evaluation Criteria
B1 Interactive Graph Visualisation	Render the attack graph in a browser using D3.js or Cytoscape.js. Critical path highlighted in red, safe nodes in green.	+5	Graph renders without error. Attack path nodes visually distinct. Zoom/pan functional. Node tooltip shows name, type, risk score.
B2 Live CVE Scoring	Integrate with NIST NVD API to auto-assign CVSS scores to Pod nodes based on container image version.	+5	API called successfully. CVSS score fetched and applied to at least one real CVE. Fallback to mock data if API unavailable. Rate limiting handled.
B3 Temporal Analysis	Store graph snapshots over time. Alert when a new attack path appears between two consecutive scans (graph diff).	+5	Two snapshots loadable. Diff identifies new nodes, new edges, and new attack paths. Alert output clearly describes what changed and the risk delta.

Section 6 — Test Case Reference Summary

ID	Algorithm	Input	Expected Output	Marks
BFS-1	Blast Radius	source=pod-webfront, hops=3	13 reachable nodes across 3 layers	3
BFS-2	Blast Radius	source=cicd-bot, hops=2	4 nodes: sa-cicd, deployer, cicd-deploy-token + 1	2
BFS-3*	Blast Radius	source=isolated-node, hops=3	0 reachable nodes, no crash	2
DIJK-1	Dijkstra	user-dev1 → db-production	5 hops, cost=24.1, CVE-2024-1234 on first edge	3
DIJK-2	Dijkstra	internet → ns-kube-system	5 hops, cost=32.0, via sa-default → cluster-admin	2
DIJK-3*	Dijkstra	no-path-src → no-path-dst	'No path found' message, exit 0	2
DFS-1	Cycle Detection	Full mock graph	1 cycle: [svc-service-a, svc-service-b]	3
DFS-2*	Cycle Detection	Hidden 3-cycle graph	Exactly 3 cycles, no duplicates	3
CNA-1	Critical Node	Full mock graph	web-frontend, 32/46 paths eliminated	5
CNA-2*	Critical Node	Hidden graph	Correct node, correct count	5

* Hidden test cases not present in mock-cluster-graph.json. Tested separately by judges.

Penalties & Disqualification Criteria

Condition	Penalty
Tool crashes or raises unhandled exception on mock data	-5 marks
Tool mutates the input cluster-graph.json file	-3 marks
Algorithm output hardcoded to match mock data (not computed)	Disqualification of that deliverable (0 marks for section)
Runtime exceeds 5 minutes on mock-cluster-graph.json (40 nodes)	-5 marks
Plagiarism or use of a pre-existing K8s security scanner as the submission	Full disqualification
Submission does not run on Python 3.10+ / Go 1.21+ without modification	-5 marks

This rubric is a synthetic reference document generated to support hackathon preparation. The official scoring rubric is distributed by the hackathon organizers. Expected outputs are derived from mock-cluster-graph.json using the reference implementation.